基于机器学习的加密代理流量识别

Performance Evaluation of Machine Learning Algorithms for Encrypted Proxy Traffic Identification

张校源（ZhangXiaoyuan）

Independent Researcher（独立研究者）

Email: silenceyuan2018@gmail.com

ORCID iD: https://orcid.org/0009-0006-6147-9256

2026年10月

摘 要

加密代理可隐藏应用层内容，但包长、到达间隔及上下行交互仍可观测。本文以公开、可核验的真实流量记录比较随机森林、支持向量机与K近邻三种分类器。JA4Tor公开流级表包含普通非Tor流量、直接Tor以及Tor经Shadowsocks、Trojan、VMess封装五类；在20 000条训练流和5 000条测试流上，仅使用30项长度、时间与方向统计量，随机森林准确率为92.12%，宏平均召回率为92.12%。MineShark公开记录包含ShadowsocksR、Trojan与VMess挖矿流量；在按原始抓包文件隔离的4 743个训练窗口与1 719个测试窗口上，支持向量机准确率为97.27%。当训练与测试的挖矿程序分别变为NBMiner和XMRig时，该值降至46.66%。这说明外在统计特征在给定数据分布内具有识别力，但对业务负载变化敏感。两套公开数据均不含可核验的VLESS与普通HTTPS完整四类标签，因此本文不报告目标四分类任务的准确率，并给出其严格复现实验条件。

关键词：加密代理流量；包长；包间到达时间；随机森林；支持向量机；数据分布变化

Abstract

Encrypted proxies conceal payloads but expose packet lengths, timing and direction. JA4Tor Fixed-split Evaluation trains on 20,000 public training flows and tests on 5,000 fixed flows (RF 92.12%; XGBoost 92.08%). Repeated Internal Evaluation resplits the training file into 16,000/4,000 flows for ten runs (RF 87.47% ± 0.39%; XGBoost 87.97% ± 0.44%). MineShark's trace-disjoint evaluation uses 4,743 training and 1,719 test windows; SVM reaches 97.27%, falling to 46.66% under NBMiner-to-XMRig workload shift. JA4Tor capture provenance is unavailable, and no external validation on CIC-Darknet2020 or ISCXVPN2016 was completed because data files were unavailable. Neither source supports the complete four-class target labels.

Keywords: encrypted proxy traffic; packet length; inter-arrival time; Random Forest; support vector machine

1 引言

Shadowsocks、VLESS与Trojan通过不同的封装和加密机制转发业务流量。被动观察者通常不能读取加密载荷，却仍能看到链路上帧或数据包的大小、发送方向与时间序列。已有研究表明，这类可见元数据能够形成加密流量指纹，但隧道封装、填充、复用和业务内容会同时改变指纹[2]。因此，问题不只是模型能否在某个数据集取得高分，还包括分数究竟反映协议、应用业务还是采集环境。

本文的目标是建立可复跑的传统机器学习基线，比较随机森林（RF）、径向基核支持向量机（SVM）与K近邻（KNN）对外在特征的利用，并检验负载迁移后性能是否保持。目标识别类别为Shadowsocks、VLESS、Trojan和普通HTTPS。由于公开数据标签与该目标并非完全一致，本文将实测任务与目标任务清楚区分：所有数值只对应数据源原有标签，不通过重命名补齐缺失类别。

2 数据来源与标签审计

首先核对了ISCXVPN2016官方说明。该数据集提供OpenVPN与非VPN流量及浏览、聊天、文件传输等应用类别，确实适合VPN识别研究，但不能据此推断Shadowsocks、VLESS或Trojan的逐协议标签[1]。本文最终采用两套包含真实采集记录且可直接下载的研究工件：NESTTOR/JA4Tor发布的流级CSV[3]，以及MineShark在Zenodo发布的抓包派生记录[4]。两者所研究的业务与协议不同，实验不混合训练。

表1 公开数据与目标标签的对应关系

JA4Tor的公开CSV训练集五类各4 000条，测试集各1000条。本文下载原文件并记录SHA-256；训练输入剔除Flow ID、IP、端口、时间戳、HTTP和TLS字段。公开CSV未提供行到原始抓包的完整关联，因而无法验证capture-disjoint；本文不作抓包隔离声明。test-cross.csv仅作为作者提供的第二测试文件使用，不推断其采集域与独立性。

MineShark归档经发布页MD5校验为6c64268ec35414fc9f483a7a4ed43755。其README明确说明ssr为ShadowsocksR，v2ray为VMess；每份TSV记录帧时间、帧长、端点和端口。本文从103份ShadowsocksR、41份Trojan、64份VMess记录提取特征，模型输入不含IP、端口、矿池名或采集日期。源工件另含normal.pcap，但其中混有大量非HTTPS端口，且没有逐流HTTPS真值标签，故不把它标为普通HTTPS。

3 特征与分类方法

图1概括从真实流量工件到分类结果的数据处理流程。JA4Tor直接读取作者提供的流级CSV；MineShark使用原始抓包派生TSV记录提取窗口统计量。文件隔离在窗口抽样之前确定，缺失值填补和标准化参数只由训练数据拟合。

图1 数据流处理与模型评估流程

3.1 可见特征

JA4Tor使用原始CSV中30项长度、时间和方向统计量：双向包数与字节数、正反向包长均值及离散程度、整流及单方向IAT均值和极值、流持续时间、下行与上行比例等。MineShark以每份采集文件内非重叠60秒窗口为样本，保留至少12帧的窗口，计算20项特征，包括帧长均值、标准差、中位数与90分位数，IAT均值、标准差与90分位数，上行帧占比、下行/上行字节比和方向切换率。设相邻包到达时间为t□，则IAT□=t□−t□□□；设下行与上行字节量分别为B↓、B↑，比例为B↓/max(B↑,1)。全部特征仅来自外在元数据。

MineShark的frame.len是链路帧长度，可能包含链路层开销和网卡卸载造成的大于常见MTU的记录；本文未将其称为纯TCP载荷长度。方向由私网客户端地址推定，但地址值只在预处理阶段使用，没有输入分类器。

3.2 模型与评价指标

RF采用200棵树、平方根特征采样与最小叶节点2；SVM采用RBF核、C=10；KNN取k=7。缺失值填补在训练折拟合，SVM和KNN再在训练折进行标准化。三种模型使用相同特征与相同测试样本，随机种子固定为42，

未针对测试集调参。RF、SVM和KNN分别代表集成树、非线性间隔学习和距离学习，便于观察外在特征的模型敏感性[5–8]。

准确率Accuracy=正确预测数/测试样本数。第c类召回率Recall□=TP□/(TP□+FN□)；宏平均召回率是各类别召回率的算术平均。另报告宏平均F1和逐类召回率，以免总体准确率掩盖少数类别失误。本文测试集经过类内等量抽样，因此部分表格中准确率与宏平均召回率相等；这属于指标定义的数学结果，不是两个独立的性能增益。

4 实验设计与可复现性

实验一分为Fixed-split Evaluation与Repeated Internal Evaluation：前者使用20 000条公开训练流和固定5 000条测试流；后者每次从公开训练文件重新划分16 000/4 000，重复十次。两组均使用同一30项特征、同一缺失值处理和同一标准化流程，但训练样本量及测试样本来源不同。Fixed-split的训练集是作者发布的train.csv，固定测试集是作者发布的test.csv；Repeated Internal不接触固定test.csv，而是在train.csv内部按类别分层抽取4 000条作为内部测试集。因而87.47% ± 0.39%回答的是“公开训练文件内部重新抽样时的稳定性”，92.12%回答的是“使用全部公开训练文件对作者固定测试文件的性能”，两者不能合并为一个平均值。实验二先按每个代理类别随机划分原始MineShark采集文件，约70%用于训练、30%用于测试，再在各分区内部按类别等量抽取窗口。这样同一文件内高度相关的窗口不会同时出现在训练和测试中。实验三从NBMiner和XMRig采集窗口中分别按类别等量抽样，前者用于训练，后者用于测试，构造更严格的挖矿业务迁移检验。

MineShark的样本数必须按实验版本区分。当前审计主结果是seed=42的抓包文件隔离划分：训练窗口为4 743，测试窗口为1 719，三类均衡后每类573。旧版results/mineshark/per_class.csv中的每类740、合计2 220，来自旧版重复实验第1次划分的历史汇总；该版本使用了不同的候选窗口抽样和逐类支持数，不能与seed=42的1 719窗口混用。当前论文的表3、混淆矩阵和逐类指标全部改用1 719窗口版本，并将2 220保留为审计记录中的历史数字，不再把它写成当前测试集。

对于JA4Tor，两组实验的预处理顺序保持一致：先删除Flow ID、地址、端口、时间戳以及HTTP/TLS字段，再对剩余统计特征进行缺失值处理；RF和树模型直接使用填补后的数值，SVM、KNN和线性模型的标准化参数只从对应训练子集拟合。固定评估的训练样本为20 000，内部重复评估每次训练样本为16 000，因此准确率差异不能归因于模型参数变化。XGBoost使用同一固定测试集和同一特征矩阵，固定评估92.08%，内部重复87.97% ± 0.44%。

所有原始下载、子集文件、特征矩阵、抓包分区表、预测结果、混淆矩阵、版本和图表均保存在项目根目录。运行脚本可重新计算论文中的每一项数值。MineShark实验评估的是给定挖矿负载和三种实际标签的识别，不能外推为一般网页访问下的四协议识别。

4.1 评估协议的证据链

为避免把文件名当成实验事实，本文为每个结果建立了从数据源到指标的证据链。JA4Tor的run_manifest记录文件哈希、列名、删除字段和模型参数；固定测试结果可由predictions.csv逐行重算，逐类support之和应等于5 000。内部重复结果由十个seed目录组成，每个目录分别保存训练行号、内部测试行号、预测结果和metrics.json，四千个测试行的逐类support之和应等于4 000。MineShark的capture_splits.csv记录每一类训练抓包数、测试抓包数、候选窗口数和最终抽样数；seed=42的最终抽样数为1 581个训练窗口/类和573个测试窗口/类，三类合计分别为4 743和1 719。这个链条使样本数、支持数和混淆矩阵可以相互校验。

审计还区分了候选窗口数与最终评估窗口数。ShadowsocksR、Trojan和VMess的候选窗口池并不相同，直接把候选池大小写成测试样本会夸大有效样本量。最终表格使用的是在每类内部等量抽样后的窗口；未被抽中的候选窗口不会进入测试指标。旧版2 220汇总之所以不能简单通过文字改成1719，是因为它的逐类支持数、候选池和抽样结果已经改变，必须把对应实验版本一起区分。

4.2 数据泄漏与相关窗口

加密流量窗口通常具有强烈的时间相关性。若先把所有窗口随机打散再划分，来自同一抓包的相邻窗口可能同时出现在训练和测试，模型可以利用采集环境、连接阶段或短期网络状态，而不必学习可迁移的代理模式。MineShark主结果先按原始抓包文件分区，再在分区内部抽样，因此至少消除了同一文件直接跨集合的泄漏路径。该设计仍不能保证窗口完全独立，因为同一客户端、同一服务器或同一批次采集仍可能共享背景条件。

JA4Tor的情况不同。公开CSV只提供流级行和标签，没有完整的行到原始PCAP或采集运行的映射。即使train.csv与test.csv在发布时看起来是两个文件，也不能从文件名推出抓包级独立性。因此本文只把它们称为固定文件划分，并把test-cross.csv称为第二公开测试文件；不把JA4Tor结果称为capture-disjoint，也不把test-cross的下降直接解释为跨节点、跨日期或跨客户端泛化。

4.3 可复现性与不确定性

单次准确率只描述一个固定划分。为评估记录选择对结果的影响，JA4Tor内部评估使用十个随机种子，报告均值和样本标准差；MineShark扩展审计也保存了十次重复的metrics和逐类结果。这里的标准差描述重新抽样协议下的波动，不是部署环境中所有可能网络的置信区间。对于按抓包分组的任务，更严格的不确定性应以抓包或采集运行作为统计单位进行分层重采样，而不是把数千个相关窗口当成完全独立样本。

模型比较遵循同一输入矩阵和同一测试集合。RF的树数量、SVM的C与核函数、KNN的邻居数在运行前固定；没有根据测试集错误反复调参。XGBoost作为扩展基线单独记录版本和运行时间，不改变原三模型表格的主结论。这样的设置牺牲了部分调参空间，但使模型之间的差异更容易解释，也避免把测试集信息泄漏到模型选择过程。

5 实验结果

5.1 JA4Tor流级基线

表2 JA4Tor五类流量分类结果 单位为百分比

图2 JA4Tor公开测试文件上的准确率与宏平均召回率

在test.csv上，RF达到92.12%的准确率，高于SVM的89.72%和KNN的88.24%。RF对Tor-SS和Tor-Trojan的召回率分别为88.60%和93.40%，反映两类代理封装流量的可区分性并不一致。换用test-cross.csv后，RF准确率降到87.54%，SVM降到76.73%，KNN降到79.40%。由于公开工件缺少抓包级对应关系，这一变化只能被视为第二测试文件上的性能差异，不能直接归因为跨节点或跨时间泛化。

5.2 MineShark抓包文件隔离评估

表3 MineShark三类代理抓包窗口分类结果 单位为百分比

当前审计工件在4 743个训练窗口和1 719个测试窗口上运行，测试集严格为三类各573个窗口；SVM最高，准确率与宏平均召回率均为97.27%，RF为94.30%，KNN为92.44%。SVM对ShadowsocksR、Trojan和VMess的召回率分别为98.60%、96.51%和96.68%。旧版结果文件results/mineshark/per_class.csv另有三类各740、合计2 220个支持数，但其对应的分区清单和抓包来源在当前工件中不存在，不能与本次1 719窗口实验合并；本文将其视为未能追溯的历史汇总，不报告其指标。与JA4Tor结果不能直接合并排名：样本单位、标签和业务负载均不同。

5.3 业务负载变化检验

表4 NBMiner训练 XMRig测试的迁移结果 单位为百分比

图3 MineShark三类代理任务的同分布评估与跨挖矿程序评估

当训练业务改为NBMiner、测试业务改为XMRig时，RF、SVM和KNN准确率分别为44.81%、46.66%和55.34%。SVM相对于按文件隔离的同分布测试下降50.61个百分点，说明它学到的部分模式与挖矿程序、底层矿池协议及连接节奏有关。该结果不意味着KNN普遍优于SVM，只说明在本次固定配置和特定迁移条件下，KNN降幅较小。

5.4 随机森林特征重要性分析

为解释JA4Tor五分类模型的判别依据，采用与5.1节完全相同的随机森林参数在20 000条训练流上重新拟合，读取基于不纯度平均下降的特征重要性（MDI）。该模型在原测试集的准确率复核为92.12%；排名仅从训练模型导出，没有使用测试集调参。全部30项重要性之和为1，图4给出前10项。

图4 JA4Tor随机森林前10项特征的MDI重要性

排名前三的特征为Bwd Packet Length Max（8.19%）、Fwd Packet Length Std（6.97%）、Down/Up Ratio（6.79%）。前10项合计占总重要性的57.18%。这表明本次模型主要利用包长分布、方向比例和部分时间统计量进行区分。MDI对连续或取值丰富的变量可能存在偏好，相关特征也会分摊重要性，因此该排序不能解释为协议的因果特征或跨网络环境的稳定排序。

6 讨论

6.1 识别力与混杂因素

加密不会自动消除帧长、时间和方向信息；两个工件的同分布实验均证明这些外在特征可用于分类。然而，代理协议与应用行为、远端服务、网络路径、捕获年份、客户端实现和TLS承载方式相互作用。MineShark不同协议文件可能使用不同的代理服务器与端口，虽然这些标识未输入模型，网络路径仍可间接影响IAT。Tor经代理封装与一般代理上网也不是同一业务分布。故高准确率不能解释为模型识别了协议的稳定指纹。

6.2 目标四分类所缺的证据

本研究不能回答“任一加密流究竟是Shadowsocks、VLESS、Trojan还是普通HTTPS”的完整四分类准确率。原因是两个已核验工件均无VLESS；MineShark的ssr必须称为ShadowsocksR，v2ray必须称为VMess；JA4Tor的NonTor不是经逐流确认的HTTPS。此外，四类别应在相同目标网站或业务组合、相近采集时段、不同节点/客户端条件下采集，按会话或采集运行分组划分，并保持测试节点与训练节点独立。只有具备这些原始PCA

P及标签后，四分类指标才具有可审查的含义。

未来扩展应在同一环境下对四种目标模式各采集多个独立节点与至少两轮时间段，保存抓包哈希、代理版本与配置、访问任务和方向定义。先按采集轮次与节点划分训练/测试，再进行RF、SVM、KNN比较，报告逐类召回率和混淆矩阵；最后加入未知代理类别和填充/复用配置检验。当前项目脚本与特征定义可作为这一扩展的基线。

6.3 相关工作与本文定位

加密流量分类的研究大致形成几条互相关联的路线。第一类工作使用包长、时间间隔和方向序列进行网站或应用指纹识别，强调即使载荷不可见，流量形状仍可能泄露信息。第二类工作研究Tor和匿名通信，关注入口、出口、网站指纹以及封装层之间的混合信号。第三类工作使用TLS握手或客户端行为形成JA3、JA4一类的指纹；这些方法与本文的流统计特征不同，但共同说明实现版本、库和配置可能成为强判别线索。第四类工作把机器学习用于VPN、恶意软件和代理流量识别，通常报告较高的同分布准确率，同时指出网络环境变化会造成明显下降。

本文不把已有方法的高准确率直接当作普适结论，而是把它们作为设计审计的依据。Panchenko等人的网站指纹研究、Hayes和Danezis的k-fingerprinting、Miller等人的HTTPS流量分析，以及Anderson和McGrew关于加密恶意流量的工作，都提示流量特征可能同时编码网站、实现、路径和业务。Tor相关研究进一步说明匿名网络中的内外层协议不能简单等同。因而本文将标签语义、抓包分组和负载迁移放在准确率之前讨论。

对于Computer Communications风格的实验报告，最重要的不是把参考文献数量堆高，而是让每个引用支持一个明确论断。经典分类器引用用于说明模型定义；Tor和网站指纹论文支持匿名通信与流量形状的背景；VPN和代理识别论文支持应用场景；跨域泛化和数据泄漏工作支持按抓包划分、重复评估和谨慎解释。JA3网页说明只作为工程背景，本文不把它当作唯一的同行评审证据。

6.4 外部数据集验证的当前状态

CIC-Darknet2020和ISCXVPN2016在本文中均属于未完成的外部验证任务。当前工作区没有取得可合法复核的数据文件，因此没有运行特征提取、模型训练、测试预测或准确率计算。摘要、结果和结论不把它们描述为已完成实验；第7节只报告现有JA4Tor和MineShark结果。任何外部准确率都必须在数据文件、许可条件、标签字典、抓包关联方式和特征兼容性完成审计后再写入。

CIC-Darknet2020的独立验证应保持其原始类别语义，例如Tor、VPN、非Tor或其他暗网相关类别，不把这些类别重命名为Shadowsocks、VLESS、Trojan或普通HTTPS。ISCXVPN2016也应按VPN与非VPN及其应用标签报告，而不能被用来填补本文目标四分类的缺失类别。只有在源数据和目标数据共享明确标签含义、特征定义和分组规则时，才可以讨论跨数据集迁移。

6.5 特征工程的解释边界

本文特征工程的目标是保留观察者在加密状态下仍能得到的信息，同时尽量排除明显的身份字段。包长度统计量描述一次流中数据单元的尺度和离散程度；IAT统计量描述发送节奏；方向比例描述上下行交互是否对称；包数、字节数和持续时间则提供流量规模。它们并不等价于协议字段，也不能单独证明某个代理实现。比如，相同的代理协议在网页浏览、视频分片和长连接消息中的包长分布可能差异很大，而不同协议在相同应用和相同网络路径下也可能产生相似的统计量。

特征选择同时具有工程和统计两方面含义。删除IP、端口和时间戳，是为了避免模型直接记住地址、服务端口或采集时间；但这些字段在真实检测系统中有时确实有用。本文删除它们并不表示它们在部署中永远不应使用，而是为了把研究问题限定为外在流量形状识别。MineShark中由私网地址推定方向，地址值只用于方向计算，不进入模型；如果客户端地址规则发生变化，方向特征也可能产生系统误差，因此复现实验必须保留方向定义和端点识别规则。

标准化是另一个容易被忽略的细节。均值和标准差如果在全体数据上计算，会把测试分布的信息带入训练过程，尤其会影响SVM和KNN的距离与核函数。本文在每个固定或重复划分中只用训练子集拟合填补器和标准化器，再把同一变换应用于测试子集。RF和XGBoost对尺度不敏感，但仍使用相同的缺失值策略，从而保证比较的差异来自分类器而不是输入清洗方式。

6.6 逐类别错误与混淆矩阵

总体准确率不能说明错误发生在哪里。MineShark固定划分中，SVM对ShadowsocksR召回率为98.60%，对Trojan为96.51%，对VMess为96.68%；三类支持数相同，所以宏平均召回率与准确率接近。RF对Trojan召回率较低而对VMess召回率较高，提示不同模型可能利用了不同的方向和时间组合。混淆矩阵应与逐类别support一起阅读：如果测试集不均衡，少数类的错误可能被总体准确率掩盖；如果测试集均衡，宏平均指标又可能比真实部署先验更乐观。本文因此同时保留逐类精确率、召回率、F1和支持数。

JA4Tor的五类标签也不能简单理解为五个独立协议。Tor-SS、Tor-Trojan和Tor-VMess表示Tor流量与外层代理封装的组合标签，NonTor则是数据源定义的非Tor类别，不能自动等同于普通HTTPS。模型把这些源标签分开，说明它能够利用当前数据中的统计差异；这不等于它能在另一组网站、节点或客户端上识别相同的协议语义。test-cross上的性能下降正是这种限制的可见信号，但由于缺少抓包来源元数据，下降原因只能保持为未确定的分布变化。

在业务迁移实验中，训练集和测试集的代理类别保持一致，但挖矿程序从NBMiner变为XMRig。SVM从同分布的97.27%下降到46.66%，表明分类边界可能高度依赖业务节奏、矿池交互或底层连接行为。这个实验不是协议分类的“失败证明”，而是一个边界条件：当应用负载改变时，原来有用的外在特征未必继续稳定。部署前应至少使用多个工作负载、多个采集批次和多个节点评估，而不能把一个固定程序上的高准确率直接推广。

6.7 统计报告与可重复使用性

论文中的数字应能够从保存的中间文件重新计算。对于每个模型，metrics.csv提供总体指标，per_class.csv提供逐类别指标，confusion矩阵提供预测与真实标签的交叉计数，predictions.csv保留逐行预测。四者之间存在简单的守恒关系：混淆矩阵所有单元之和应等于测试样本数，每一行或每一列的合计应与逐类支持数相符，正确分类单元之和除以总数应等于准确率。这样的交叉检查比只保存一张结果表更可靠，也能及时发现筛选条件或标签顺序被误改。

实验脚本还记录运行环境、随机种子、输入文件哈希和模型参数。随机种子不是统计独立性的保证，但可以让一次实验在相同软件环境下重现。软件版本变化可能影响树模型的随机实现、缺失值处理或浮点计算，因此正式复核时应优先使用run_manifest记录的版本，并把重新运行得到的差异报告为复现偏差，而不是静默覆盖原始结果。

论文不把十次重复的均值误写成部署置信区间。重复内部评估改变了训练和测试行的选择，反映的是公开训练文件内部的抽样敏感性；它不能代替不同日期、不同客户端、不同网络路径和不同采集者的外部验证。要估计更接近部署的变化，应以抓包或采集运行作为重采样单位，并在每次重采样中重新拟合全部预处理步骤。当前结果为这一后续设计提供基线，但没有越过数据证据的边界。

6.8 对实际检测系统的启示

如果把本文模型用于实际网络监测，第一步不是选择准确率最高的分类器，而是定义检测对象和允许的错误类型。对于安全告警，误报会增加分析负担，漏报则可能使代理流量逃避发现；不同场景的代价不同。模型应输出类别概率、拒识或未知状态，而不是强迫所有流量进入训练过的三类或五类。特别是，当流量来自未知工作负载、填充策略、复用连接或新的代理实现时，开放集错误可能比闭集准确率更重要。

第二步是把数据治理纳入模型生命周期。每次采集应保存抓包哈希、时间、节点、客户端版本、代理配置、访问任务、标签生成规则和许可状态。训练、验证和测试必须按抓包或运行分组，避免把同一会话的相邻窗口拆到不同集合。数据更新后，应分别报告旧模型在新采集上的结果和重新训练模型的结果，以区分模型老化、特征变化和标签变化。本文对MineShark的文件级划分以及对JA4Tor证据不足的保留，正是为了让这种治理要求可执行。

第三步是报告失败条件。与其只给出单一最高分，不如同时给出固定文件结果、重复内部结果、跨抓包结果和跨工作负载结果，并说明每个结果使用的训练样本、测试样本和预处理参数。这样读者可以判断模型适用于哪一种采集条件，也可以决定是否需要重新采集数据。本文的主要贡献因此不是宣称某种算法解决了加密代理识别，而是提供一套可检查的比较框架，明确哪些结论已经由数据支持，哪些仍需要新的外部实验。

6.9 校准、拒识与跨域比较

准确率只考察最终类别是否正确，不能说明模型的置信度是否可信。对于加密流量监测，测试流可能来自训练集之外的代理、工作负载或网络路径，模型即使给出较高的最大类别概率，也可能只是把未知样本强行归入最相近的已知类别。后续实验应在独立校准集上评估可靠性图、期望校准误差和Brier分数，并设置拒识阈值。拒识样本应单独统计，而不能简单算作某一已知类别的错误。

跨数据集比较还需要统一样本单位和标签语义。JA4Tor以流为单位，MineShark以60秒窗口为单位；一个流和一个窗口承载的时间范围、相关性和标签生成方式不同。即使两个实验都报告准确率，也不能把它们排列成同一张排行榜。CIC-Darknet2020和ISCXVPN2016未来若加入，应分别报告其原始任务，说明类别映射、窗口化规则、抓包分组、特征缺失和训练规模，再决定是否可以进行有限的跨域分析。

本文还存在若干已知限制。第一，公开数据的采集条件不是由本文控制，无法补充缺失的客户端版本、节点位置或完整会话标签。第二，MineShark的窗口虽然按抓包文件隔离，但同一采集批次仍可能共享环境因素。第三，JA4Tor没有抓包关联信息，固定文件划分不能证明源抓包独立。第四，外部数据文件未取得，不能评估新数据上的性能。第五，传统模型参数是固定基线，不代表经过系统调参后的最优结果。明确这些限制，是为了让高分结果保持正确的适用范围。

统计显著性在本文中只作辅助解释。固定测试集上的百分点差异没有经过独立重复试验，不能据此宣称某模型在所有部署条件下显著优于另一模型。重复内部评估的标准差可以显示记录抽样带来的波动，但十次运行仍不是对所有网络环境的完整抽样。因而本文优先报告原始样本数、划分规则和逐类混淆矩阵，让读者自行判断差异大小是否具有实际意义，并把需要更多采集轮次的比较列为后续工作。

结果复核还应关注标签顺序和单位换算。混淆矩阵的行列顺序、百分比的小数位以及窗口和流的单位都必须在表题中写明；否则相同数字可能被误读为不同实验。本文统一保留原始计数，并在百分比之外说明总样本数，确保图表、正文和脚本输出可以互相追溯。

7 结论

结果显示JA4Tor固定划分与十次内部重划分是两种不同协议：RF分别为92.12%和87.47% ± 0.39%，XGBoost分别为92.08%和87.97% ± 0.44%，不可混用。MineShark当前可追溯的按抓包隔离测试集为1719窗口、573/类，SVM准确率97.27%，跨挖矿程序测试降至46.66%。JA4Tor抓包来源元数据不足以支持capture-disjoint声明，CIC-Darknet2020及ISCXVPN2016外部验证尚未完成。现有结果不证明目标四分类性能。

参考文献

[1] Drapper Gil G, Lashkari A H, Mamun M, et al. Characterization of Encrypted and VPN Traffic Using Time-Related Features[C]. ICISSP, 2016: 407–414. 数据说明：https://www.unb.ca/cic/datasets/vpn.html

[2] Xue D, Kallitsis M, Houmansadr A, et al. Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes[C]. 33rd USENIX Security Symposium, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/xue-fingerprinting

[3] Zhou C, Huang C, Zhang Q, et al. NESTTOR: Detecting Tor-over-Proxy Traffic with Factorized Decisions[J]. IEEE Transactions on Network and Service Management, 2026. DOI: 10.1109/TNSM.2026.3743261. 数据：https://github.com/das-lab/JA4Tor

[4] Xi S, Fu T, Bu K, et al. MineShark: Cryptomining Traffic Detection at Scale[C]. NDSS, 2025. DOI: 10.14722/ndss.2025.230402. 工件：https://zenodo.org/records/13624058

[5] Breiman L. Random Forests[J]. Machine Learning, 2001, 45: 5–32. DOI: 10.1023/A:1010933404324.

[6] Cortes C, Vapnik V. Support-vector networks[J]. Machine Learning, 1995, 20: 273–297. DOI: 10.1007/BF00994018.

[7] Cover T, Hart P. Nearest neighbor pattern classification[J]. IEEE Transactions on Information Theory, 1967, 13(1): 21–27. DOI: 10.1109/TIT.1967.1053964.

[8] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[9] Panchenko A, Lanze F, Zinnen A, et al. Website Fingerprinting at Internet Scale[C]. NDSS, 2016. DOI: 10.14722/ndss.2016.23477.

[10] Hayes J, Danezis G. k-fingerprinting: A Robust Scalable Approach to Website Fingerprinting[C]. USENIX Security, 2018: 1187–1203.

[11] Miller B, Huang L, Dyer K P, et al. I Know Why You Went to the Clinic: Risks and Realization of HTTPS Traffic Analysis[C]. PETS, 2014, 2014(1): 143–163. DOI: 10.1515/popets-2014-0012.

[12] AlSabah M, Briffa J A, et al. A Statistical Analysis of Tor Traffic for Website Fingerprinting[C]. ACM CCS, 2016. DOI: 10.1145/2976749.2989057.

[13] Anderson B, McGrew D. Machine Learning for Encrypted Malware Traffic Classification[C]. IEEE MILCOM, 2016: 1–6. DOI: 10.1109/MILCOM.2016.7795421.

[14] Anderson B, McGrew D. Identifying Encrypted Malware Traffic with Contextual Flow Data[C]. Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, 2016: 35–46. DOI:

10.1145/2996758.2996760.

[15] JA3: SSL/TLS Client Fingerprinting. Salesforce Engineering, 2017. 该方法的同行评审基础见 Anderson and McGrew [13,14]；本文不将网页说明作为唯一证据。

附录 复现说明

实验环境为Python 3.12与scikit-learn 1.9.1。运行python scripts/run_ja4tor_baseline.py生成实验一的表格、逐类指标与图2；运行python scripts/run_mineshark_proxy_baseline.py和python scripts/run_mineshark_workload_shift.py生成实验二、三，再运行python scripts/plot_workload_shift.py生成图3。原始工件、下载与提取脚本位于data/raw和scripts，各实验的run_manifest.json及capture_splits.csv记录版本、哈希、特征和划分。重新下载MineShark归档可运行python scripts/download_mineshark.py，其来源和MD5已写入源清单。

数据仓库：JA4Tor，https://github.com/das-lab/JA4Tor；MineShark，https://zenodo.org/records/13624058。

特征重要性与方法图：运行python scripts/analyze_feature_importance.py重建；完整排名位于results/ja4tor/feature_importance.csv。

复现代码仓库：https://github.com/silenceyuana/encrypted-proxy-traffic-identification

生成式人工智能工具使用声明

本研究使用OpenAI Codex辅助文稿起草、中英文表述与润色、实验代码编写、图表绘制和排版检查。模型训练、评估及特征重要性数值由本地Python/scikit-learn程序实际计算，AI生成的文字不作为实验数据来源。作者应在投稿前审核全部文字、引用、代码与结果，并对最终稿的准确性、原创性及研究诚信承担责任。

数据源	实有类别	样本与来源	与目标四类的差距
JA4Tor	NonTor、Tor、Tor-SS、Tor-Trojan、Tor-VMess	20 000训练流；5 000测试流；另有5 024条test-cross	无VLESS；NonTor不能确认为纯HTTPS；代理内载荷为Tor
MineShark	ShadowsocksR、Trojan、VMess	208个原始采集文件；本文提取12 401个60秒窗口	ShadowsocksR不等于Shadowsocks；无VLESS和HTTPS

测试文件	模型	准确率	宏召回率	宏F1
test.csv	Random Forest	92.12	92.12	92.27
test.csv	SVM RBF	89.72	89.72	89.90
test.csv	KNN k=7	88.24	88.24	88.21
test-cross.csv	Random Forest	87.54	87.69	87.33
test-cross.csv	SVM RBF	76.73	76.99	74.38
test-cross.csv	KNN k=7	79.40	79.58	78.23

模型	训练窗口	测试窗口	准确率	宏召回率	宏F1
Random Forest	4743	1719	94.30	94.30	94.30
SVM RBF	4743	1719	97.27	97.27	97.27
KNN k=7	4743	1719	92.44	92.44	92.43

模型	训练窗口	测试窗口	准确率	宏召回率	宏F1
Random Forest	3702	1899	44.81	44.81	46.08

模型	训练窗口	测试窗口	准确率	宏召回率	宏F1
SVM RBF	3702	1899	46.66	46.66	42.87
KNN k=7	3702	1899	55.34	55.34	54.55